Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3701551.3703504

RESEARCH-ARTICLE

Self-supervised Time-aware Heterogeneous Hypergraph Learning for Dynamic Graph-level Classification

MALIK KHIZAR HAYAT, Macquarie University, Sydney, NSW, Australia

SHAN XUE, Macquarie University, Sydney, NSW, Australia

JIA WU, Macquarie University, Sydney, NSW, Australia

BILAL KHAN, Macquarie University, Sydney, NSW, Australia

JIAN YANG, Macquarie University, Sydney, NSW, Australia

Open Access Support provided by:

Macquarie University

PDF Download
3701551.3703504.pdf
20 January 2026
Total Citations: 2
Total Downloads: 845

Published: 10 March 2025

Citation in BibTeX format

WSDM '25: The Eighteenth ACM International Conference on Web Search and Data Mining
March 10 - 14, 2025
Hannover, Germany

Conference Sponsors:
SIGKDD
SIGMOD
SIGIR
SIGWEB

WSDM '25: Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining (March 2025)
https://doi.org/10.1145/3701551.3703504
ISBN: 9798400713293

Self-supervised Time-aware Heterogeneous Hypergraph Learning for Dynamic Graph-level Classification

Malik Khizar Hayat*
malik.khizar@hdr.mq.edu.au
Macquarie University
Sydney, Australia

Shan Xue
emma.xue@mq.edu.au
Macquarie University
Sydney, Australia

Jia Wu
jia.wu@mq.edu.au
Macquarie University
Sydney, Australia

Bilal Khan
bilal.khan@mq.edu.au
Macquarie University
Sydney, Australia

Jian Yang
jian.yang@mq.edu.au
Macquarie University
Sydney, Australia

Abstract

Graph-level learning has gained significant importance in understanding complex systems, such as social and biological networks, but often fails to capture evolving and multi-entity interaction. Current dynamic graph-level classification methods struggle with evolving structures, global properties, and high-order interactions. To address these challenges, we propose DyH$_2$GNet, a novel time-aware self-supervised heterogeneous hypergraph neural network that models complex, non-pairwise interactions and their temporal dynamics by integrating k-hop nodes and attribute correlations into a heterogeneous hypergraph. It features a temporal embedding method that captures high-order proximity and the dynamic context of heterogeneous interactions through intra- and inter-snapshot attention. Furthermore, a graph-level pooling layer aggregates node features with temporal context, supported by self-supervised learning that leverages temporal contrastive learning to maximize the use of unlabeled data. Experiments on real-world datasets demonstrated significant improvements in capturing dynamic graph-level features through unsupervised graph-level tasks, including graph similarity ranking, anomaly detection, and trend analysis.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**.

Keywords

heterogeneous hypergraph neural networks, temporal dynamic, graph-level representation, self-supervised learning

ACM Reference Format:
Malik Khizar Hayat, Shan Xue, Jia Wu, Bilal Khan, and Jian Yang. 2025. Self-supervised Time-aware Heterogeneous Hypergraph Learning for Dynamic Graph-level Classification. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining (WSDM '25), March 10–14, 2025, Hannover, Germany*. ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3701551.3703504

*Corresponding author

This work is licensed under a Creative Commons Attribution International 4.0 License.

WSDM '25, Hannover, Germany
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1329-3/25/03
https://doi.org/10.1145/3701551.3703504

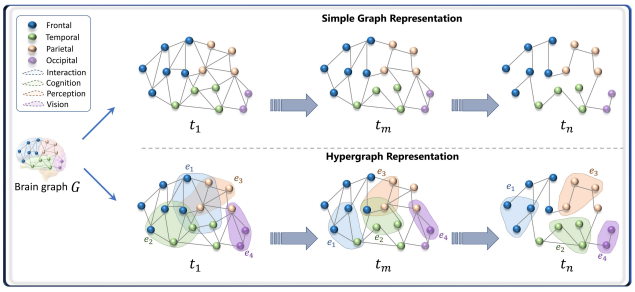

Figure 1: A toy example illustrates the temporal evolution of a brain graph G, capturing Alzheimer's disease progression across early (t_1), mid (t_m), and late (t_n) stages. At t_1, hyperedge e_1 (Interaction) represents a robust cognitive network across brain lobes; e_2 (Cognition) captures memory processing; e_3 (Perception) groups sensory interpretation nodes; e_4 (Vision) links visual processing nodes. By t_m, e_1 and e_3 show reduced connectivity, and e_4 remains stable but isolated. By t_n, all hyperedges exhibit substantial disintegration, with e_1 and e_4 persisting in a diminished state, underscoring the brain's diminished cognitive and visual capacities.

1 Introduction

Traditional graph representations often fall short in capturing the full complexity of the systems, such as social networks [25, 40], where the overall structure can reveal community behaviors, and biological networks [18, 36], where the global interactions between genes or proteins can indicate overall cellular function, especially when these interactions encompass collective behavior across multiple entities. This limitation is particularly pronounced in dynamic settings where relationships within the graph evolve over time. For example, during a pandemic, health social networks [6, 48] form a complex web of interactions among healthcare providers, government agencies, community groups, and the public, affecting both information spread and the effectiveness of interventions. Similarly, in biological systems, understanding the dynamics of fluctuating functional connectivity between brain regions [21] or the progression of diseases like Alzheimer [33] requires capturing complex, non-pairwise interactions that traditional models often simplify.

Hypergraphs [9] are powerful to model these complex, non-pairwise interactions. Nodes involved in various types of hyperedges are exposed to diverse information and resources, enhancing

the accuracy of predictions and the generalization of dynamic graph-level tasks [17]. For instance, in a social network, a node with high temporal connectivity across different social roles or groups can act as a central or bridging figure. Likewise, in biological networks, genes or proteins that interact across various pathways or functions consistently are often important for understanding complex diseases or multi-functional biological processes [19]. Figure 1 illustrates the difference between simple graphs and hypergraphs, highlighting how hyperedges (e_1 to e_4) capture the intricate connections and high-order interactions essential for cognitive functions in brain lobes, whereas simple graphs often miss these complex relationships, leading to an incomplete understanding of the system.

Graph-level learning [34, 44] has increasingly become a focal point in understanding complex systems, setting itself apart from node- or edge-level learning [17, 46], by capturing holistic properties and behaviors of entire networks. Existing approaches for graph classification face several challenges and can be categorized into three types: (1) Static graph-level embedding methods, which are designed for static contexts and struggle to adapt when graph structures evolve; (2) Temporal node-level embedding methods, which update embeddings at the node level but fail to capture the overall graph structure and crucial high-order interactions; and (3) Dynamic graph-level embedding methods, which aim to address graph dynamics but often rely on simplified pairwise interactions, failing to manage complex, high-order interactions effectively.

These limitations significantly hinder achieving precise high-order data representation for dynamic graph-level classification [4], including challenges in modeling heterogeneous interactions, preserving structural and semantic integrity [47], and utilizing unlabeled data [37] to enhance classification performance. To address these, we propose a novel Time-aware Self-supervised Dynamic Heterogeneous HyperGraph Neural Network (DyH$_2$GNet), introducing a temporal hypergraph embedding that captures both high-order proximity and dynamic context of heterogeneous interactions, a graph-level pooling layer that aggregates both structural and semantic contexts, and a self-supervised learning (SSL) strategy that employs temporal contrastive learning to leverage unlabeled data, as illustrated in Figure 2. In summary of contributions, DyH$_2$GNet introduces a novel framework for dynamic graph-level classification using hypergraphs enabling the integration and analysis of complex and high-order interactions, develops an SSL strategy that utilizes time-aware contrastive learning to effectively leverage unlabeled data in a dynamic setting, and demonstrates the applicability and effectiveness across multiple domains, achieving an overall average improvement of 16.88% across key evaluation metrics, highlighting its versatility and robustness in handling complex dynamic data.

2 DyH$_2$GNet

2.1 Preliminaries and Problem Statement

Definition 2.1. ***(Hypergraph)*** Given a hypergraph $G = (V, E, \boldsymbol{W})$, V and E represent the set of nodes and hyperedges, respectively, and $\boldsymbol{W} \in \mathbb{R}^{|E|\times|E|}$ denotes a diagonal matrix with weights of hyperedges. G can be succinctly described by an incidence matrix $\boldsymbol{H} \in \{0,1\}^{|V|\times|E|}$. A hyperedge $e \in E$ connects two or more nodes.

Definition 2.2. ***(Heterogeneous Hypergraph)*** Given a heterogeneous hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{P}, \mathcal{E}, \mathcal{Q}, \boldsymbol{W})$, $\mathcal{V}$ denotes the set of heterogeneous nodes in different types of $\mathcal{P}$, $\mathcal{E}$ represents the set of hyperedges in various types of $\mathcal{Q}$, and $\boldsymbol{W} \in \mathbb{R}^{|\mathcal{E}|\times|\mathcal{E}|}$ is the hyperedge weight matrix. The hypergraph is considered heterogeneous when $|\mathcal{P}| + |\mathcal{Q}| > 2$. The relationships between nodes and hyperedges can be captured by an incidence matrix $\boldsymbol{H}$, where $\boldsymbol{H}(v, e) = 1$ if the node v is part of the hyperedge e, and 0 otherwise.

The degree of a node $v \in \mathcal{V}$, which is the count of hyperedges incident to it, is represented by the node degree matrix $\boldsymbol{D}_v$, where $\boldsymbol{D}_v(i,i) = \sum_{e\in\mathcal{E}} \boldsymbol{W}(e)\boldsymbol{H}(i,e)$. Similarly, the degree of a hyperedge $e \in \mathcal{E}$, which is the number of nodes within it, is represented by the hyperedge degree matrix $\boldsymbol{D}_e$, where $\boldsymbol{D}_e(i,i) = \sum_{v\in\mathcal{V}} \boldsymbol{H}(v,i)$.

Setting in a heterogeneous environment, $\deg^h(v)$ denotes the heterogeneous node degree of v, defined by the number of unique hyperedge types in which v participates.

$$\deg^h(v) = |\{q \in \mathcal{Q} : \exists e \in \mathcal{E}_q, H(v,e) = 1\}| \tag{1}$$

where $\mathcal{E}_q$ represents the q type's subset of hyperedges, and $H(v,e)$ is the incidence matrix indicating the participation of v in e.

Definition 2.3. ***(Dynamic Heterogeneous Hypergraph)*** Given $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{P}, \mathcal{E}_t, \mathcal{Q}, \boldsymbol{W}_t)$ for each timestamp t, a dynamic heterogeneous hypergraph captures the evolution of nodes $\mathcal{V}_t$ and hyperedges $\mathcal{E}_t$, with $\boldsymbol{H}_t$ dynamically representing their connections, where $\boldsymbol{H}_t(v,e) = 1$ if v is part of e at t and 0 otherwise.

Definition 2.4. ***(Structural Hyperedge)*** In a dynamic heterogeneous hypergraph $\mathcal{G}_t$, a structural hyperedge $e_t^{\text{str}} \in \mathcal{E}_t$ consists of a subset of nodes from $\mathcal{G}_t$, linked by k-hop interactions. Formally, $e_t^{\text{str}} = \{v_1, v_2, \ldots, v_k\}$ where each $v_i \in \mathcal{V}_t$, and $1 \le k \le |\mathcal{V}_t|$. The weight of e_t^{str} is denoted by $w_{e_t^{\text{str}}}$ and is stored as a diagonal element in the matrix $\boldsymbol{W}_t^{\text{str}} \in \mathbb{R}^{|\mathcal{V}_t|\times|\mathcal{V}_t|}$. The collection of structural hyperedges at time t is represented by $\mathcal{E}_t^{\text{str}} \subset \mathcal{E}_t$.

Definition 2.5. ***(Behavioral Hyperedge)*** In a dynamic heterogeneous hypergraph $\mathcal{G}_t$, a behavioral hyperedge $e_t^{\text{sem}} \in \mathcal{E}_t$ consists of a subset of nodes from $\mathcal{G}_t$, linked by attribute correlations. Formally, $e_t^{\text{sem}} = \{v_1, v_2, \ldots, v_l\}$ where each $v_i \in \mathcal{V}_t$, and $1 \le l \le |\mathcal{V}_t|$. The weight of e_t^{sem} is denoted by $w_{e_t^{\text{sem}}}$ and is recorded as a diagonal element in the matrix $\boldsymbol{W}_t^{\text{sem}} \in \mathbb{R}^{|\mathcal{V}_t|\times|\mathcal{V}_t|}$. The collection of behavioral hyperedges at time t is represented by $\mathcal{E}_t^{\text{sem}} \subset \mathcal{E}_t$.

Definition 2.6. ***(Temporal Connectivity)*** For $v_i \in \mathcal{V}_t$ in $e_i \in \mathcal{E}_t$, $D(v_i)_t$ across hyperedge types over t is defined as:

$$D(v_i)_t = \frac{\sum_{t'<t} \deg^h(v_i)_{t'} \cdot (w_{v_i})_{t'}}{|\mathcal{Q}|} \tag{2}$$

where $(w_{v_i})_{t'}$ is a learnable weight for v_i's interactions at t' (attention-derived), and $|\mathcal{Q}|$ is the total number of hyperedge types.

Problem 1. ***Self-supervised Time-aware Heterogeneous Hypergraph Learning.*** *Given a sequence of dynamic graph snapshots $\mathbb{G} = \{\mathrm{G}_1, \mathrm{G}_2, \ldots, \mathrm{G}_T\}$, where each snapshot $\mathrm{G}_t = (\mathrm{V}_t, \mathrm{E}_t, \boldsymbol{X}_t)$ captures the state of the graph at a particular timestamp t, we aim to learn a mapping function $g_\theta : \mathrm{G}_t \to \mathbb{R}^d$ that transforms each graph G_t into a low-dimensional representation $\mathbf{x}^{\mathrm{G}_t} \in \mathbb{R}^d$, with d being the dimensionality of the final embeddings. The aim is to model the temporal dynamics of graph evolution and generate representations for each graph snapshot that encapsulate both structural and behavioral characteristics of the data. These representations are intended for use in dynamic graph-level downstream tasks over time.*

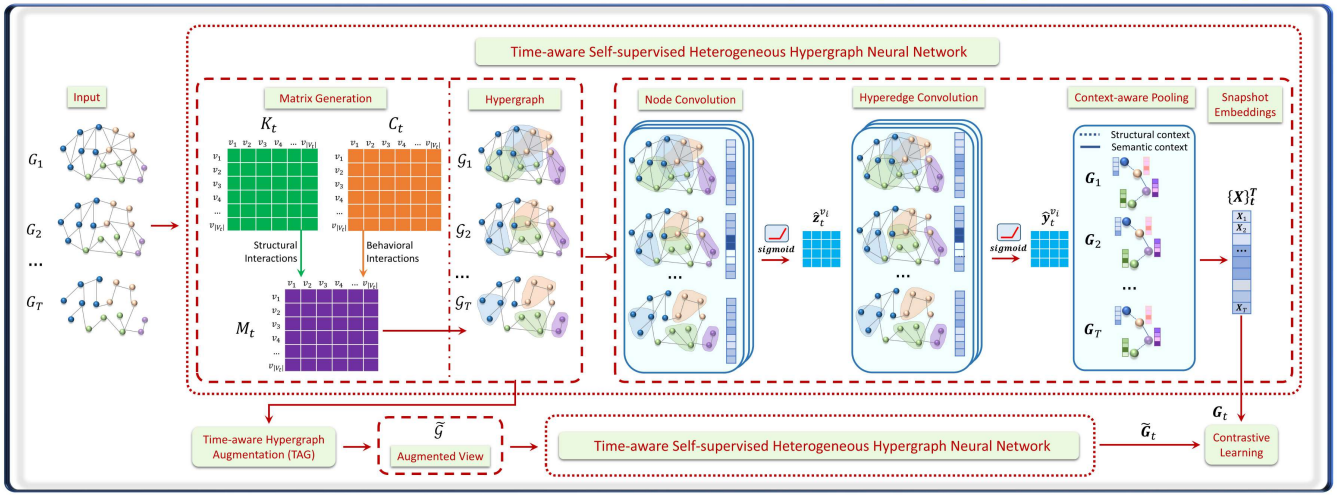

Figure 2: In DyH$_2$GNet framework, a snapshot G_t ($t = 1, \cdots, T$) is first captured by structural interactions in K_t and behavioral interactions in C_t, combined in matrices M_t, and constructed into a heterogeneous hypergraph $\mathcal{G}_t$. The framework then performs multi-typed node convolution for $\hat{z}_t^{v_i}$, hyperedge convolution for $\hat{y}_t^{v_i}$, and a graph-level pooling to refine $\{G_t\}$ into the output $\{X\}_t^T$. It applies a time-aware hypergraph augmentation to produce an augmented hypergraph view $\tilde{\mathcal{G}}_t$ for the graph-level contrastive learning between G_t and $\tilde{G}_t$.

2.2 Hypergraph Construction

While methods like k-Nearest Neighbors [9] and k-Means [32] have been used to form hyperedges, they often fail to capture complex relationships in evolving networks. We combine the network-based k-hop [18] strategy and behavioral correlations to ensure better adaptability to changing network structures and node behaviors.

2.2.1 Structural Interactions. To capture structural interactions among nodes in dynamic graphs, for each node $v_i \in \mathrm{V}_t$, the k-hop neighborhood is defined by a symmetric matrix $\boldsymbol{K}_t \in \mathbb{R}^{|\mathrm{V}_t| \times |\mathrm{V}_t|}$, where $|\mathrm{V}_t|$ denotes the total nodes in G_t. For each t, a matrix $\boldsymbol{K}_t$ is constructed, with each element $\boldsymbol{K}_{ij}$ defined as[1]:

$$\boldsymbol{K}_{ij} = \begin{cases} 1, & \text{if length}(v_i, v_j) \leq k \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

where length$(\cdot)$ is the shortest path between nodes v_i, v_j in V_t.

2.2.2 Behavioral Interactions. For behavioral similarities, a Spearman's correlation matrix $\boldsymbol{C}_t$ is calculated at each t for each pair of node feature vectors $\boldsymbol{v}_i$ and $\boldsymbol{v}_j$. This quantifies the strength and direction of associations between nodes over time, offering insights into how changes at one point might influence future states. The min-max normalized correlation, as in Eq. (4), reveals behavioral patterns that structural connectivity alone cannot capture.

$$\boldsymbol{C}_t = [C_{ij}] = \left[\frac{C_{ij} - C_{min}}{C_{max} - C_{min}} \right] \tag{4}$$

To combine the structural interaction matrix $\boldsymbol{K}$ and the behavioral interaction matrix $\boldsymbol{C}$ for hypergraph construction, we create a matrix $\boldsymbol{M}$. In dynamic networks, using a fixed correlation value may miss subtle changes in connectivity. Instead, applying a dynamic threshold based on the mean correlation in $\boldsymbol{C}$ for each graph snapshot better adapts to network changes. This approach ensures $\boldsymbol{M}$ captures both connectivity and emphasizes important behavioral connections. For each t, $\boldsymbol{M}_t = [\boldsymbol{M}_{ij}]$ is defined with

$$\boldsymbol{M}_{ij} = \begin{cases} 1, & \text{if } \boldsymbol{K}_{ij} = 1 \text{ and } \boldsymbol{C}_{ij} \geq \text{mean}(\boldsymbol{C}) \\ 0, & \text{otherwise} \end{cases} \tag{5}$$

where each row in $\boldsymbol{M}$ represents a hyperedge in $\mathcal{G}_t$.

2.3 Heterogeneous Hypergraph Convolution

Temporal convolutions allow the model to learn from sequence of hypergraph snapshots, enabling it to capture patterns across multiple timestamps. The convolution for $\mathcal{G}_t$ can be expressed as:

$$\boldsymbol{X}_t^{l+1} = \sigma\left(\boldsymbol{P}_t\left(\boldsymbol{H}_t \boldsymbol{W}_{v_t}^l \boldsymbol{H}_t^\top \boldsymbol{X}_t^l + \boldsymbol{H}_t \boldsymbol{W}_{e_t}^l \boldsymbol{H}_t^\top \boldsymbol{X}_t^l\right)\right) \tag{6}$$

where $\boldsymbol{X}_t^l$ denotes node features at convolutional layer l, $\boldsymbol{P}_t$ is the learnable weight matrix between layers l and $l+1$, $\boldsymbol{H}_t$ represents the incidence matrix, and $\boldsymbol{W}_{v_t}^l$ and $\boldsymbol{W}_{e_t}^l$ are the weight matrices for nodes and hyperedges at t. To normalize this operation, Eq. (6) can be written in its row-normalized form as:

$$\boldsymbol{X}_t^{l+1} = \sigma(\boldsymbol{P}_t(\boldsymbol{D}_{v_t}^{-1} \boldsymbol{H}_t \boldsymbol{W}_{v_t}^l \boldsymbol{D}_{e_t}^{-1} \boldsymbol{H}_t^\top \boldsymbol{X}_t^l + \boldsymbol{D}_{v_t} \boldsymbol{H}_t \boldsymbol{W}_{e_t}^l \boldsymbol{D}_{e_t} \boldsymbol{H}_t^\top \boldsymbol{X}_t^l) \tag{7}$$

where $\boldsymbol{D}_{v_t}$ and $\boldsymbol{D}_{e_t}$ are node and hyperedge degree matrices at t.

In a heterogeneous hypergraph, hyperedges connect nodes in various types, representing diverse relationships. By aggregating information from node features–first from node to hyperedge, then from hyperedge to node–DyH$_2$GNet creates dual attention on $\mathcal{G}_t$: (1) ***Intra-snapshot attention*** captures structural dynamics by focusing on interactions within a single timestamp, and (2) ***Inter-snapshot attention*** captures temporal dynamics by attending to interactions across timestamps, leveraging temporal connectivity.

2.3.1 Multi-typed Node Convolution. To capture type-specific interactions and incorporate temporal aspects into the node convolution process, we introduce a temporal attention mechanism

[1]The subscript t is omitted for simplicity, but all matrices ($\boldsymbol{K}$, $\boldsymbol{C}$, and $\boldsymbol{M}$) are recalculated for each t.

that considers the influence of past node states on current representations. Employing self-attention mechanism of transformers [24, 27, 42], the attention-enhanced node convolution operation aggregates features from each node and its neighbors, (hyperedges from $\boldsymbol{M}$), facilitating node-to-hyperedge information flow at each timestamp. Using self-attention [42], we compute attention coefficients and learn weights for multi-typed nodes. For a given node $v_i \in \mathcal{V}_t$ at t, and its neighbors from $\boldsymbol{M}_{i*}$ (where the ith row in $\boldsymbol{M}$ represents v_i's neighbors), the importance of node pair (v_i, v_j) is determined through a combination of structural (intra-snapshot) and temporal (inter-snapshot) attention, resulting in the aggregated node embeddings $\hat{z}_t^{v_i}$. Finally, we transform the input at t to the linear projections (query ($\boldsymbol{Q}$), key ($\boldsymbol{K}$), and value ($\boldsymbol{V}$)) [42] as:

$$\boldsymbol{Q}_t^{v_i} = \boldsymbol{W}_Q \boldsymbol{z}_t^{v_i}, \quad \boldsymbol{K}_t^{v_j} = \boldsymbol{W}_K \boldsymbol{z}_t^{v_j}, \quad \boldsymbol{V}_t^{v_j} = \boldsymbol{W}_V \boldsymbol{z}_t^{v_j}$$

where $\boldsymbol{W}_Q$, $\boldsymbol{W}_K$, and $\boldsymbol{W}_V$ are learnable weight matrices. Next, from $\boldsymbol{M}_t$, the importance of node pair (v_i, v_j) with projected feature vectors $(\hat{\boldsymbol{v}}_i, \hat{\boldsymbol{v}}_j)$ can be defined as:

$$v_{i,j}^{\boldsymbol{M}_t^e} = \sigma_{\mathcal{P}} \left(\hat{\boldsymbol{v}}_{\boldsymbol{M}_t^e}^{\intercal} \cdot [\boldsymbol{Q}_t^{v_i} \| \boldsymbol{K}_t^{v_j}] \boldsymbol{M}_{i*} \right) \tag{8}$$

where $\sigma_{\mathcal{P}}$ is the activation function for multi-typed node convolution, $\hat{\boldsymbol{v}}_{\boldsymbol{M}_t^e}$ the attention vector, and the softmax function is:

$$\text{softmax}_j \left(v_{i,j}^{\boldsymbol{M}_t^e} \right) = \frac{\exp\left(\sigma \left(\hat{\boldsymbol{v}}_{\boldsymbol{M}_t^e}^{\intercal} \cdot [\boldsymbol{Q}_t^{v_i} \| \boldsymbol{K}_t^{v_j}] \right) \right)}{\sum\limits_{x \in \boldsymbol{M}_{i*}} \exp\left(\sigma \left(\hat{\boldsymbol{v}}_{\boldsymbol{M}_t^e}^{\intercal} \cdot [\boldsymbol{Q}_t^{v_i} \| \boldsymbol{K}_t^{v_x}] \right) \right)} \tag{9}$$

$$\alpha_t^{struct} = \text{softmax}_j \left(v_{i,j}^{\boldsymbol{M}_t^e} \right) \tag{10}$$

where σ is the activation function, $\|$ is the concatenation operation.

The normalized weight vectors of neighboring nodes $\boldsymbol{V}_t^{v_j}$ are aggregated to update the v_i's structural embedding $\boldsymbol{z}_t^{struct}$ at t, incorporating structural attention scores α_t^{struct}.

$$\boldsymbol{z}_t^{struct} = \sum_{v_j \in \mathcal{N}_t^{v_i}} \alpha_t^{struct} \boldsymbol{V}_t^{v_j} \tag{11}$$

It allows v_i to integrate relevant information from its neighborhood $\mathcal{N}_t^{v_i}$ within the same snapshot at t.

For temporal attention, the input is linearly projected as follows.

$$\boldsymbol{Q}_t^{v_i} = \boldsymbol{W}_Q \boldsymbol{z}_t^{struct}, \quad \boldsymbol{K}_{t'}^{v_j} = \boldsymbol{W}_K \boldsymbol{z}_{t'}^{v_j}, \quad \boldsymbol{V}_{t'}^{v_j} = \boldsymbol{W}_V \boldsymbol{z}_{t'}^{v_j}$$

The temporal importance of (v_i, v_j) can be defined in Eq. (12) and a normalized form in Eqs. (13)–(14):

$$v_{i,j}^{\boldsymbol{M}_t^e} = \sigma_{\mathcal{P}} \left(\hat{\boldsymbol{v}}_{\boldsymbol{M}_t^e}^{\intercal} \cdot [\boldsymbol{Q}_t^{v_i} \| \boldsymbol{K}_{t'}^{v_j}] + D(v_i)_t \right) \tag{12}$$

$$\text{softmax}_j \left(v_{i,j}^{\boldsymbol{M}_t^e} \right) = \frac{\exp\left(\sigma \left(\hat{\boldsymbol{v}}_{\boldsymbol{M}_t^e}^{\intercal} \cdot [\boldsymbol{Q}_t^{v_i} \| \boldsymbol{K}_{t'}^{v_j}] \right) \right)}{\sum\limits_{x \in \boldsymbol{M}_{i*}} \sum\limits_{t' \le t} \exp\left(\sigma \left(\hat{\boldsymbol{v}}_{\boldsymbol{M}_t^e}^{\intercal} \cdot [\boldsymbol{Q}_t^{v_i} \| \boldsymbol{K}_{t'}^{v_x}] \right) \right)} \tag{13}$$

$$\alpha_t^{temp} = \text{softmax}_j \left(v_{i,j}^{\boldsymbol{M}_t^e} \right) \tag{14}$$

Similarly, the value weight vectors $\boldsymbol{V}_{t'}^{v_j}$ are aggregated to obtain the temporally attended embedding $\boldsymbol{z}_t^{temp}$ for v_i at t incorporating the temporal information in α_t^{temp}.

$$\boldsymbol{z}_t^{temp} = \sum_{v_j \in \mathcal{N}_t^{v_i}} \sum_{t' \le t} \alpha_t^{temp} \boldsymbol{V}_{t'}^{v_j} \tag{15}$$

After applying the node convolution at t, the node embedding can be represented as:

$$\hat{\boldsymbol{z}}_t^{v_i} = \boldsymbol{z}_t^{temp} \tag{16}$$

2.3.2 Multi-typed Hyperedge Convolution. The attention mechanism is applied during hyperedge convolution to capture the influence of multi-typed hyperedges, dynamically aggregating type-specific information. Since a node v_i can participate in multiple hyperedges (denoted by $\mathcal{B}_t^{v_i}$), we learn hyperedge weights based on their features. Incorporating embeddings from the node convolution step, we define transformation vectors as:

$$\boldsymbol{Q}_t^{v_i} = \boldsymbol{W}_Q \hat{\boldsymbol{z}}_t^{v_i}, \quad \boldsymbol{K}_t^{v_j} = \boldsymbol{W}_K \boldsymbol{z}_t^{v_j}, \quad \boldsymbol{V}_t^{v_j} = \boldsymbol{W}_V \boldsymbol{z}_t^{v_j}$$

To obtain the significance of hyperedge $e_i \in \mathcal{E}_t$, we apply attention on v_i given the query vectors incorporating $\mathcal{B}_t^{v_i}$ as:

$$v_{i,j}^{\boldsymbol{M}_t^e} = \sigma_Q \left(\sum_{e \in \mathcal{B}_t^{v_i}} \left(\hat{\boldsymbol{v}}_{\boldsymbol{M}_t^e}^{\intercal} \cdot [\boldsymbol{Q}_t^{v_i} \| \boldsymbol{K}_t^{v_j}] \right) \right) \tag{17}$$

Using softmax, Eq. (17) can be normalized as:

$$\text{softmax}_j \left(v_{i,j}^{\boldsymbol{M}_t^e} \right) = \frac{\exp\left(\sigma \left(\hat{\boldsymbol{v}}_{\boldsymbol{M}_t^e}^{\intercal} \cdot [\boldsymbol{Q}_t^{v_i} \| \boldsymbol{K}_t^{v_j}] \right) \right)}{\sum\limits_{e \in \mathcal{B}_t^{v_i}} \sum\limits_{x \in \mathcal{N}(v_i, e)} \exp\left(\sigma \left(\hat{\boldsymbol{v}}_{\boldsymbol{M}_t^e}^{\intercal} \cdot [\boldsymbol{Q}_t^{v_i} \| \boldsymbol{K}_t^{v_x}] \right) \right)} \tag{18}$$

where $x \in \mathcal{N}(v_i, e)$ represents v_i's neighborhood in hyperedge e.

$$\beta_t^{struct} = \text{softmax}_j \left(v_{i,j}^{\boldsymbol{M}_t^e} \right) \tag{19}$$

By aggregating value vectors of neighboring nodes within t,

$$\boldsymbol{y}_t^{struct} = \sum_{v_j \in \mathcal{N}_t^{v_i}} \beta_t^{struct} \boldsymbol{V}_t^{v_j} \tag{20}$$

Following attention, normalization, and aggregation from 2.3.1, temporal attention for hyperedge convolution is represented as:

$$\boldsymbol{y}_t^{temp} = \frac{1}{Q+1} \left(\sum_{q \in Q} \sum_{t' \le t} \beta_{t,q}^{temp} \boldsymbol{V}_{t'}^{v_j} \right) \tag{21}$$

where $\beta_{t,q}^{temp}$ represents attention for all hyperedges and Q represents all hyperedge types. After applying the hyperedge convolution at t, the node embedding can be represented as:

$$\hat{\boldsymbol{y}}_t^{v_i} = \boldsymbol{y}_t^{temp} \tag{22}$$

2.4 Attentive Graph-level Pooling

For each graph snapshot, we learn node representations based on structural (k-hops) and semantic (correlations) within hyperedges. Nodes carry type information through structural connections and correlational information through semantic relationships. For graph-level pooling, we use the attention mechanism to learn the structural and semantic importance. To manage complexity, our attention-based pooling layer prioritizes the most relevant parts of the graph, dynamically assessing each node's contribution to the final graph-level representations. Attention scores for structural ($\alpha_t^{v_i}$) and semantic ($\beta_t^{v_i}$) contexts are computed at each t as follows:

$$\alpha_t^{v_i} = \exp\left(\tanh\left(\hat{\boldsymbol{y}}_t^{v_i} \cdot \boldsymbol{W}_t^{\text{str}} \right) \right) \tag{23}$$

$$\beta_t^{v_i} = \exp\left(\tanh\left(\hat{\boldsymbol{y}}_t^{v_i} \cdot \boldsymbol{W}_t^{\text{sem}} \right) \right) \tag{24}$$

We then use an attentive pooling layer to aggregate the node embeddings into graph-level embeddings for each hypergraph snapshot.

$$G_t = \sum_{v_i \in \mathcal{V}_t} (\alpha_t^{v_i} + \beta_t^{v_i}) \hat{\boldsymbol{y}}_t^{v_i} \tag{25}$$

By using a weighted sum of the embeddings with their corresponding attention weights, the attentive pooling layer gives more significance to important nodes, enhancing its ability to capture comprehensive, dynamic graph-level features for each snapshot. This approach provides a detailed temporal view of the evolving graph.

2.5 Contrastive Learning

Traditional graph contrastive learning often focuses on static graphs, altering the graph structure but overlook underlying relationships. Although they perform well for tasks such as node classification and link prediction in static settings, they fail to capture temporal dependencies and evolving dynamics. To address this, we propose a high-order-aware dual adaptive augmentation for dynamic heterogeneous hypergraphs, which balances attention between observed structures and inherent relationships for dynamic graph-level tasks.

2.5.1 Time-aware Graph Augmentation. Traditional augmentations include feature-based methods that mask portions of node/edge attributes, structure-based methods that add or remove nodes/edges, and sampling-based methods that select nodes/edges under specific rules. In dynamic settings, augmentation must consider both the temporal dynamics and the multi-typed relationships among nodes. Temporal dynamics reflect how nodes and their relationships evolve, making it essential to assess the temporal importance of nodes at specific timestamps. Each node and hyperedge in a heterogeneous hypergraph has distinct semantic importance that should be preserved during augmentation. Removing a node in such a hypergraph not only alters immediate connections but also affects long-term and type-specific relationships, which are vital for dynamic graph-level tasks. To preserve high-order interactions in hyperedge $e_i \in \mathcal{E}_t$ in $\mathcal{G}_t$, we propose a dual adaptive Time-aware hypergrAph auGmentation (TAG). TAG combines *structural* importance in node degrees with *temporal* importance $\alpha_t^{v_i}$ to determine the node dropout probability $P_t^{v_i}$ for a node $v_i \in e_i$ at time t.

$$P_t^{v_i} = \lambda \cdot (1 - \alpha_t^{v_i}) + (1 - \lambda) \cdot \frac{1}{1 + (\deg^h(v_i)_t)} \tag{26}$$

where λ is a weighting parameter for structural and temporal terms.

Nodes less connected to different types of hyperedges, deemed less critical, are assigned a higher dropout probability, which helps eliminate insignificant connections and strengthens the representation's robustness. Conversely, nodes with a higher heterogeneous degree, considered more significant, are given a lower dropout probability, preserving key relationships and essential semantics. Similarly, nodes with lower temporal importance receive a higher dropout probability, while those with higher temporal importance get a lower dropout probability. We collect the nodes selected for removal into the set $\mathcal{R}_t$ by combined dropout probabilities.

After node removal, we update the hyperedge weights by considering both the importance of the remaining nodes and the impact of the removed ones. Specifically, the updated hyperedge weight $\bar{w}_t^{e_i}$ is calculated using the node importance from Eq. (11) based on the importance α for $v_r \in \mathcal{R}_t$ in relation to v_i as follows.

$$\bar{w}_t^{e_i} = \frac{1}{|e_i|} \left(w_t^{e_i} \sum_{v_r \in \mathcal{R}_t} (1 - \alpha_t^{v_i, v_r}) + \sum_{v_j \in e_i \setminus \mathcal{R}_t} \alpha_t^{v_i, v_j} \right) \tag{27}$$

where $w_t^{e_i}$ is the original weight of e_i, $\alpha_t^{v_i, v_r}$ represents the importance of the removed nodes $\mathcal{R}_t$ for v_i, and $\alpha_t^{v_i, v_j}$ represents the importance of the remaining nodes in e_i, providing a robust representation of t-evolving hypergraph.

GAT generates the augmented view $\tilde{\mathcal{G}} = \{\tilde{\mathcal{G}}_1, \tilde{\mathcal{G}}_2, \ldots, \tilde{\mathcal{G}}_T\}$ from the original view $\mathcal{G} = \{\mathcal{G}_1, \mathcal{G}_2, \ldots, \mathcal{G}_T\}$, where $\mathcal{G}_t$ is the hypergraph snapshot at time t. Leveraging the aforementioned attention mechanism, the initial graph embeddings $\boldsymbol{G}_t$ for each snapshot $\mathcal{G}_t$ are refined by aggregating information from neighbors across both views. For each snapshot $\mathcal{G}_t$:

$$\boldsymbol{G}_t^{org} = \text{attention}(\boldsymbol{G}_t^0, \mathcal{G}_t) \tag{28}$$

$$\boldsymbol{G}_t^{aug} = \text{attention}(\boldsymbol{G}_t^0, \tilde{\mathcal{G}}_t) \tag{29}$$

2.5.2 Temporal Contrastive Learning. After obtaining the refined graph embeddings, $\boldsymbol{G}_t^{org}$ and $\boldsymbol{G}_t^{aug}$, we employ contrastive learning across original and augmented views. Nodes within the same graph are considered positive pairs, while nodes across different graphs and those in at least one hyperedge are treated as negative samples. This approach ensures the model effectively distinguishes positive pairs even among similar samples. Given the temporal nature of the hypergraphs, the negative samples set for v_i at t is defined as:

$$\mathscr{V}_t^{org} = \{j \mid (i, j) \in \mathcal{E}_{\mathcal{G}_t}\}, \quad \mathscr{V}_t^{aug} = \{j \mid (i, j) \in \mathcal{E}_{\tilde{\mathcal{G}}_t}\}$$

where $\mathcal{E}_{\mathcal{G}_t}$ and $\mathcal{E}_{\tilde{\mathcal{G}}_t}$ are the hyperedge sets in both original and augmented views at t. The InfoNCE contrastive loss objective for v_i at t is given as:

$$\mathcal{L}_{v_i,t}^{org} = -\log \frac{P_t}{P_t + S_t^{org} + S_t^{aug}} \tag{30}$$

$$\mathcal{L}_{v_i,t}^{aug} = -\log \frac{\bar{P}_t}{\bar{P}_t + \bar{S}_t^{org} + \bar{S}_t^{aug}} \tag{31}$$

where $P_t = \exp\left(\mathrm{m}(\boldsymbol{g}_t^{v_i}, \tilde{\boldsymbol{g}}_t^{v_j})/\tau\right)$, $S_t^{org} = \sum_{j \in \mathscr{V}_t^{org}} \exp\left(\mathrm{m}(\boldsymbol{g}_t^{v_i}, \boldsymbol{g}_t^{v_j})/\tau\right)$, and $S_t^{aug} = \sum_{j \in \mathscr{V}_t^{aug}} \exp\left(\mathrm{m}(\boldsymbol{g}_t^{v_i}, \tilde{\boldsymbol{g}}_t^{v_j})/\tau\right)$. Likewise, $\mathcal{L}_t^{aug}$ can be defined with the target embedding from $\boldsymbol{G}_t^{aug}$. The contrastive loss objective is given as:

$$\mathcal{L} = \frac{1}{T} \sum_{t=1}^{T} \left(\frac{1}{|\mathcal{V}_t|} \sum_{v_i \in \mathcal{V}_t} \left(\mathcal{L}_{v_i,t}^{org} + \mathcal{L}_{v_i,t}^{aug} \right) \right) \tag{32}$$

where $|\mathcal{V}_t|$, the number of nodes at time t, normalizes the loss according to the graph size at that specific snapshot.

Algorithm 1 presents the overall process of DyH$_2$GNet. The major steps are hypergraph construction (lines 3-6), node attention, and hyperedge attention (lines 8-15), attentive graph-level pooling (lines 17-19), and time-aware hypergraph augmentation (line 20). The time complexity for Algorithm 1 is $\mathcal{O}(|T| \cdot (|\mathcal{E}| + |\mathcal{V}| + |T|))$, where T is the number of total timestamps.

Algorithm 1: DyH$_2$GNet

Input: $\{G_1, G_2, \ldots, G_T\}$: Set of dynamic graph snapshots, where $G_t = (V_t, E_t, \boldsymbol{X}_t)$; k
Output: $\boldsymbol{X} = \{\boldsymbol{X}_t\}_{t=1}^{T} = \{\boldsymbol{X}_1, \boldsymbol{X}_2, \ldots, \boldsymbol{X}_T\}$

Initialize embeddings $\{\boldsymbol{X}_t\}_{t=1}^{T} = \{\}$
for $t \in \{1, \ldots, T\}$ **do**
  Generate $\boldsymbol{K}_t$ using k via Eq. (3) (structural interactions)
  Generate $\boldsymbol{C}_t$ via Eq. (4) (behavioral interactions)
  Generate $\boldsymbol{M}_t$ using Eq. (5)
  Construct $\mathcal{G}_t$ using $\boldsymbol{K}_t$, $\boldsymbol{C}_t$, and $\boldsymbol{M}_t$
  for $v_i \in \mathcal{V}_t$ **do**
    Intra-snapshot node attention via Eqs. (8)–(10)
    $\boldsymbol{z}_t^{struct} \leftarrow$ Update $\boldsymbol{z}_t^{v_i}$ via Eq. (11)
    Inter-snapshot node attention via Eqs. (12)–(14)
    $\boldsymbol{z}_t^{temp} \leftarrow$ Update $\boldsymbol{z}_t^{v_i}$ via Eq. (15)
    Intra-snapshot hyperedge attention via Eqs. (17)–(19)
    $\boldsymbol{y}_t^{struct} \leftarrow$ Update $\boldsymbol{y}_t^{v_i}$ via Eq. (20)
    Inter-snapshot hyperedge attention from Sec. 2.3.1
    $\boldsymbol{y}_t^{temp} \leftarrow$ Update $\boldsymbol{y}_t^{v_i}$ via Eq. (21)
  end
  $\alpha_t^{v_i} \leftarrow$ Structural context via Eq. (23)
  $\beta_t^{v_i} \leftarrow$ Semantic context via Eq. (24)
  $\boldsymbol{G}_t \leftarrow$ Update graph embeddings via Eq. (25)
  $\tilde{\boldsymbol{G}}_t \leftarrow$ TAG via Eqs. (26)–(27)
  Compute $\mathcal{L}_{v_i,t}^{org}$ via Eq. (30)
  Compute $\mathcal{L}_{v_i,t}^{aug}$ via Eq. (31)
  $\mathcal{L} \leftarrow$ Compute contrastive loss via Eq. (32)
  Update parameters via backpropagation
  Embeddings for t: $\boldsymbol{X}[t] = \boldsymbol{G}_t$
end
return $\boldsymbol{X}$

3 Experiments and Discussion

3.1 Datasets

Table 1 summarizes the statistics of five large real-world datasets [4] used in the experimentation, including social network *Reddit Game of Thrones* (GOT), *Reddit Formula1* (Formula1), *Facebook wall posts* (Facebook), and *Slashdot*, where users are nodes and interactions form edges; and communication network *Enron*, which represents email exchanges, with employees as nodes and different types of email communications, such as direct emails and CCs, as edges. Temporal complexity is measured by the average time interval between each snapshot and its most similar prior snapshot using Maximum Common Subgraph (MCS) [5].

Table 1: Dataset Statistics

Datasets	# of Nodes	# of Edges	# of Snapshots	Granularity	Temporal Complexity
GOT	156,732	834,753	62	Daily	4.67
Formula1	38,702	254,731	61	Daily	13.30
Facebook	46,873	857,815	30	Monthly	5.56
Slashdot	51,083	140,778	13	Monthly	2.88
Enron	87,062	1,146,800	182	Weekly	3.45

3.2 Baselines

The proposed method, DyH$_2$GNet, is compared with baselines from three categories. Static graph-level embedding methods include Graph2vec [29], Sub2vec [1], and UGraphEmbed [2]. Dynamic node-level embedding methods consist of Node2vec [15], GAE [22], SDNE [38], DynamicTriad [49], DynGEM [14], DynAE [13], EvolveGCN [30], MTSN [26], and TREND [41]. Finally, dynamic graph-level embedding methods include tgGraphEmbed [4], DGLE [20], and GraphERT[3]. For dynamic node-level methods, we apply static graph-level embedding to each graph snapshot and select the best-performing aggregation [16], either mean or max-pooling.

3.3 Setups

To construct the hypergraph for dynamic graph-level classification task, we apply a generalized two-fold approach that incorporates both structural and behavioral interactions (as described in the Sec. 2.2). For social network datasets, k-hop neighborhoods are defined based on user interactions (posts, comments, and replies). For the communication network dataset, k-hop captures email communication links among employees. Behavioral interactions are modeled using a correlation matrix $\boldsymbol{C}$, derived from features such as user interaction frequency and betweenness centrality, which identifies key nodes that act as bridges within the network. At each t, we compute the Spearman's correlation between nodes' feature vectors and normalize these values to capture behavioral similarities, forming the matrix $\boldsymbol{M}$ from $\boldsymbol{K}$, and $\boldsymbol{C}$ to construct the heterogeneous hypergraph to train DyH$_2$GNet. Convolutional layers share the same receptive fields, with hyperparameters initialized via Xavier initialization [12] and optimized using the Adam optimizer [28] at a 0.005 learning rate. The embedding dimension is set to 128, with dropout applied at a rate of $p = 0.5$ to prevent overfitting. Experiments for DyH$_2$GNet[2] were conducted on an NVIDIA GeForce RTX 3080 GPU using the PyTorch Geometric framework.

3.4 Experimental Results

3.4.1 Graph Similarity Ranking. Graph similarity ranking is a critical task that evaluates a model's ability to capture evolving structures and relationships, particularly in dynamic graphs. Temporal graph similarity ranking identifies graph snapshots most similar to a given one, regardless of their chronological proximity. This supports applications like detecting coordinated influence operations on social media and uncovering organized efforts to strategically boost key agendas through dynamic share/reply network analysis.

To evaluate DyH$_2$GNet, we first generate embeddings for all snapshots of the given datasets and rank each snapshot G_t against other $G_o (o \neq t)$ using cosine similarity $\cos(\boldsymbol{X}_t, \boldsymbol{X}_o)$. We compute correlation and precision scores using metrics: Precision at 10 (P@10), Precision at 20 (P@20), Spearman's (ρ), and Kendall's (τ) correlation. Table 2 reports results averaged over 10 runs, ensuring robustness in evaluation. Our method consistently outperforms baselines, except on Slashdot dataset for the P@5 metric. On GOT and Formula 1, DyH$_2$GNet shows substantial improvements in both precision metrics, with an average 28.67% increase in P@10 and P@20 compared to the best baselines. On Facebook, DyH$_2$GNet achieves a 17.83% average improvement in precision metrics. On

[2]https://github.com/khizer-hayat/DyH2GNet

Table 2: Comparison of graph similarity ranking results with baselines

Methods	GOT				Formula1				Facebook				Slashdot				Enron			
	P@10	P@20	τ	ρ	P@10	P@20	τ	ρ	P@10	P@20	τ	ρ	P@5	P@10	τ	ρ	P@10	P@20	τ	ρ
Static graph-level																				
Graph2vec	0.260	0.381	0.038	0.056	0.169	0.320	0.043	0.063	0.423	0.713	0.120	0.176	0.292	0.800	0.026	0.045	0.045	0.059	-0.033	-0.046
Sub2vec	0.278	0.416	0.046	0.068	0.238	0.370	0.026	0.039	0.750	0.871	0.355	0.452	0.462	0.900	0.215	0.271	0.168	0.269	0.110	0.150
UGraphEmbed	0.160	0.355	0.022	0.039	0.182	0.300	-0.030	-0.040	0.353	0.685	0.012	0.021	0.385	0.808	0.037	0.074	0.073	0.137	0.028	0.044
Dynamic node-level																				
Node2vec	0.336	0.431	0.069	0.103	0.214	0.361	0.047	0.083	0.680	0.840	0.303	0.414	0.538	0.908	0.229	0.306	0.379	0.452	0.107	0.139
GAE	0.235	0.342	0.044	0.066	0.200	0.342	0.036	0.062	0.613	0.820	0.292	0.337	0.492	0.885	0.168	0.227	0.277	0.360	0.118	0.156
SDNE	0.352	0.457	0.120	0.197	0.262	0.388	0.044	0.078	0.400	0.645	0.095	0.120	0.415	0.885	0.095	0.124	0.316	0.400	0.087	0.138
DynamicTriad	0.277	0.364	0.131	0.195	0.243	0.396	0.024	0.033	0.733	0.818	0.271	0.395	0.646	0.869	0.201	0.276	0.322	0.425	0.112	0.153
DynGEM	0.340	0.411	0.075	0.113	0.192	0.339	0.029	0.045	0.356	0.733	0.094	0.115	0.569	0.915	0.245	0.315	0.335	0.377	0.103	0.143
DynAE	0.192	0.357	0.019	0.030	0.229	0.397	0.009	0.012	0.389	0.743	0.122	0.163	0.473	0.900	0.002	0.025	0.069	0.145	0.009	0.012
EvolveGCN	0.211	0.254	0.059	0.096	0.257	0.319	0.023	0.056	0.703	0.791	0.295	0.309	0.553	0.693	0.034	0.087	0.431	0.501	0.067	0.101
MTSN	0.162	0.195	0.104	0.115	0.203	0.296	0.017	0.043	0.297	0.398	0.087	0.105	0.213	0.344	0.012	0.043	0.116	0.211	0.029	0.091
TREND	0.295	0.311	0.089	0.109	0.226	0.280	0.031	0.045	0.723	0.793	0.254	0.337	0.263	0.498	0.021	0.041	0.274	0.365	0.081	0.112
Dynamic graph-level																				
tdGraphEmbed	0.355	0.457	0.160	0.232	0.274	0.400	0.060	0.092	0.750	0.892	0.398	0.522	**0.785**	0.915	0.347	0.463	0.385	0.489	0.127	0.188
DGLE	0.435	0.481	0.177	0.272	0.265	0.410	0.076	0.106	0.806	0.896	0.447	0.559	0.723	0.885	0.400	0.524	0.479	0.532	0.172	0.251
GraphERT	0.415	0.487	0.065	0.103	0.277	0.345	0.023	0.045	0.810	0.843	0.313	0.376	0.698	0.734	0.365	0.497	0.463	0.492	0.185	0.211
DyH$_2$GNet	**0.516**	**0.573**	**0.196**	**0.358**	**0.354**	**0.493**	**0.092**	**0.119**	**0.882**	**0.991**	**0.567**	**0.611**	0.783	**0.925**	**0.411**	**0.577**	**0.575**	**0.611**	**0.215**	**0.301**

Slashdot, DyH$_2$GNet performs comparably in P@10 and correlation metrics but slightly underperforms in P@5 due to the limited number of timestamps ($T = 13$), which affects the k-hop strategy's ability to learn temporal patterns effectively. On Enron, DyH$_2$GNet shows significant improvements, with an average 22.5% increase in precision metrics. Enhancements in Spearman's (ρ) and Kendall's (τ) correlation further validate its effectiveness in capturing dynamic graph-level properties. ***Overall***, DyH$_2$GNet demonstrates significant improvements across datasets, effectively modeling complex interactions and temporal dynamics in graph-level tasks, highlighting its robustness and generalizability across diverse networks.

3.4.2 Anomaly Detection. Anomaly detection in dynamic graphs involves identifying significant structural deviations over time, with real-world applications such as improving financial predictions and aiding in risk management by detecting anomalies in social networks, which can correlate with stock market movements. In DyH$_2$GNet, after generating the representations for all snapshots, the difference between consecutive timestamps G_t and G_{t+1} is defined as the cosine distance between their embeddings $\boldsymbol{X}_t$, and $\boldsymbol{X}_{t+1}$ as $\Delta G_t = 1 - \cos(\boldsymbol{X}_t, \boldsymbol{X}_{t+1})$. Anomalies are identified by sorting these differences and selecting the top K. We evaluate this task on the GOT and Formula1, where anomalies are defined by the air dates of episodes and race dates, respectively, due to activity spikes. Performance is measured using Mean Reciprocal Rank (MRR), chosen for its effectiveness in ranking significant changes in dynamic environments. The results in Table 3 show that DyH$_2$GNet outperforms top baselines with an MRR of 0.441 on GOT (a 19.2% improvement) and 0.619 on Formula1 (a 21.8% improvement). These gains stem from DyH$_2$GNet's ability to capture collective behaviors via hyperedges, grouping interactions around major events and improving anomaly detection in dynamic graphs.

3.4.3 Trend Analysis. The trend analysis task aims to find trends in evolutionary networks by correlating graph embeddings over time with real-world data, such as Google Trends, which tracks the popularity of search queries. By comparing our graph embeddings with Google Trends, we can assess how effectively our method captures trends in dynamic graphs. In this experiment, we use Spearman's correlation (ρ) to measure the strength and direction of the monotonic relationship between two ranked variables, providing a robust metric for trend analysis. DyH$_2$GNet is applied to GOT and Formula1 datasets, using Google Trends as the ground truth for public interest. The results in Table 3 show high ρ values, demonstrating DyH$_2$GNet's effectiveness in capturing trends. Specifically, our method achieves a ρ of 0.873 on GOT (an 11.6%

Table 3: Comparison of anomaly detection and trend analysis results with baselines

Methods	GOT		Formula1	
	MRR	ρ	MRR	ρ
Static graph-level				
Graph2vec	0.332	0.647	0.136	0.424
Sub2vec	0.106	0.534	0.115	0.413
UGraphEmbed	0.364	0.365	0.036	0.252
Dynamic node-level				
Node2vec	0.370	0.559	0.129	0.449
GAE	0.056	0.630	0.327	0.585
SDNE	0.352	-0.336	0.037	-0.070
DynamicTriad	0.239	0.634	0.425	0.424
DynGEM	0.281	0.134	0.085	0.502
DynAE	0.072	-0.098	0.027	-0.396
EvolveGCN	0.061	0.279	0.191	-0.112
MTSN	0.062	0.031	0.061	0.270
TREND	0.041	-0.458	0.051	0.233
Dynamic graph-level				
tdGraphEmbed	0.355	0.751	0.508	0.485
DGLE	0.095	0.165	0.056	0.148
GraphERT	0.370	0.782	0.521	0.556
DyH$_2$GNet	**0.441**	**0.873**	**0.619**	**0.701**

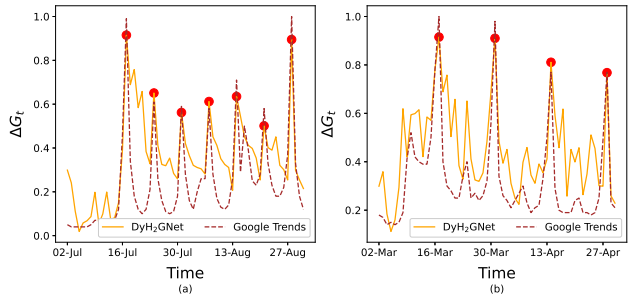

Figure 3: Trend Analysis (a) GOT, and (b) Formula1

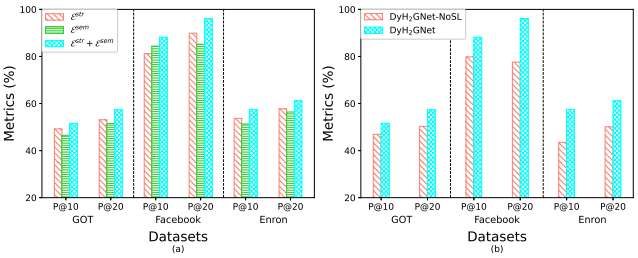

Figure 4: Ablation anlaysis (a) hypergraph construction, and (b) self-supervised learning

improvement) and 0.701 on Formula1 (a 20.8% improvement) over the best performing baselines. These improvements highlight the effectiveness of DyH$_2$GNet in capturing the rich temporal dynamics and high-order relationships. For instance, to Formula1 dataset, our method accurately captures the rise and fall of user discussions correlated with race dates, showcasing the influence of collective user behavior. DyH$_2$GNet effectively cluster user interactions, enabling it to identify subtle yet significant patterns that reveal trends and predict their future development. Figure 3 illustrates the ΔG_t values from DyH$_2$GNet against Google Trends for (a) GOT and (b) Formula1 datasets (normalized), where ΔG_t in (a) correlates with episode podcasting dates and (b) spikes with the increase of users engaging in discussions around race events. Red dots highlight the capability to capture and predict real-world trends.

3.5 Ablation Analysis

The ablation study evaluates DyH$_2$GNet's hypergraph construction and SSL contributions to graph similarity ranking.

3.5.1 Impact of Hypergraph Construction. We first evaluate the model performance upon different interaction types, including structural ($\mathcal{E}^{str}$), behavioral ($\mathcal{E}^{sem}$), and their combination ($\mathcal{E}^{str} + \mathcal{E}^{sem}$), highlighting the importance of high-order interactions in capturing complex relationships within dynamic graphs. Figure 4(a) demonstrates that combined interactions yields higher precision in graph similarity ranking across all datasets. DyH$_2$GNet employs k-hop neighborhoods for structural interactions and node correlation for behavioral interactions to construct a heterogeneous hypergraph. The experiment further identifies that a k-range of 3 to 4 effectively captures complex social interactions in datasets like Facebook and Enron, while a k-range of 2 to 3 is better suited for the simpler structural patterns observed in the Reddit GOT dataset.

3.5.2 Impact of SSL. The experiment also evaluates the impact of the SSL on the performance of DyH$_2$GNet by comparing it to a variant that excludes the SSL component, named DyH$_2$GNet-NoSL. Figure 4(b) shows DyH$_2$GNet consistently outperforming DyH$_2$GNet-NoSL in graph similarity ranking. This underscores the significance of SSL in learning nuanced representations and detecting subtle graph similarities.

4 Related Works

Heterogeneous graph representation learning addresses multi-typed nodes and edges. Traditional meta-path-based methods [7, 10] face challenges due to their reliance on domain knowledge and computational inefficiency. While attention- and SSL-based methods [23, 35] capture node features for graph-level embeddings, they miss high-order information by focusing on pairwise interactions.

Hypergraphs capture non-pairwise relationships and are used in areas like social recommendation and bioinformatics [33, 43]. HGNN [9] and its variant HGNN$^+$ [11] handle high-order data but are limited to homogeneous static hypergraphs. While recent work [18] explores hypergraphs for heterogeneous graphs, these models remain static, unlike our dynamic graph-level representations.

Dynamic graph representation learning focuses on capturing evolving structures in temporal graphs. DynamicTriad [49] and EvolveGCN [30] model temporal changes but struggle with high-order interactions and heterogeneous data types. Recent advancements, including tdGraphEmbed [4], DGLE [20], and GraphERT [3], better handle dynamic changes, they still fall short in capturing the rich, heterogeneous nature of real-world graphs.

Graph contrastive learning, leveraging SSL, optimizes mutual information between data views [8]. GCC [31] and GraphCL [45] focus on node and subgraph-level representations, while HeCo [39] and HeGCL [34] apply cross-view comparisons for heterogeneous graphs. However, these methods often fail to incorporate complex structural and semantic details in heterogeneous graphs.

5 Conclusion

In this paper, we introduce a novel self-supervised time-aware heterogeneous hypergraph neural network for dynamic graph-level classification. Our approach leverages intra- and inter-snapshot attention to enhance dynamic hypergraph representations to better capture both structural and semantic complexities over time. Extensive experimentation on real-world datasets demonstrates the superiority of proposed method in handling dynamic graph-level tasks, particularly in the context of social and communication networks. Developing time-aware, hypergraph-based methods for multi-graph-level learning, capable of capturing temporal evolution across multiple graphs, is the primary focus of our future work.

Acknowledgments

This work was supported by the Australian Research Council (ARC) under Projects LP210301259 and DP230100899, the MQ Research Acceleration Project (MQRAS) 334481525, Macquarie University Data Horizons Research Center, and the Applied AI Research Center.

References

[1] Bijaya Adhikari, Yao Zhang, Naren Ramakrishnan, and B Aditya Prakash. 2018. Sub2vec: Feature learning for subgraphs. In *Advances in Knowledge Discovery and Data Mining: 22nd Pacific-Asia Conf., PAKDD 2018, Melbourne, VIC, Australia, June 3-6, 2018, Proceedings, Part II 22*. Springer, 170–182.

[2] Yunsheng Bai, Hao Ding, Yang Qiao, Agustin Marinovic, Ken Gu, Ting Chen, Yizhou Sun, and Wei Wang. 2019. Unsupervised inductive graph-level representation learning via graph-graph proximity. *arXiv preprint arXiv:1904.01098* (2019).

[3] Moran Beladev, Gilad Katz, Lior Rokach, Uriel Singer, and Kira Radinsky. 2023. GraphERT–Transformers-based Temporal Dynamic Graph Embedding. In *Proceedings of the 32nd ACM Intl. Conf. on Info. and Knowledge Management*. 68–77.

[4] Moran Beladev, Lior Rokach, Gilad Katz, Ido Guy, and Kira Radinsky. 2020. tdgraphembed: Temporal dynamic graph-level embedding. In *Proceedings of the 29th ACM Intl. Conf. on Information & Knowledge Management*. 55–64.

[5] Horst Bunke and Kim Shearer. 1998. A graph distance metric based on the maximal common subgraph. *Pattern recognition letters* 19, 3-4 (1998), 255–259.

[6] Ke Chen, Ying Weng, Akram A Hosseini, Tom Dening, Guokun Zuo, and Yiming Zhang. 2024. A comparative study of GNN and MLP based machine learning for the diagnosis of Alzheimer's Disease involving data synthesis. *Neural Networks* 169 (2024), 442–452.

[7] Yuxiao Dong, Nitesh V Chawla, and Ananthram Swami. 2017. metapath2vec: Scalable representation learning for heterogeneous networks. In *Proceedings of the 23rd ACM SIGKDD Intl. Conf. on Knowledge Discovery and Data Mining*. 135–144.

[8] Boxin Du, Changhe Yuan, Robert Barton, Tal Neiman, and Hanghang Tong. 2022. Self-supervised Hypergraph Representation Learning. In *2022 IEEE Intl. Conf. on Big Data (Big Data)*. IEEE, 505–514.

[9] Yifan Feng, Haoxuan You, Zizhao Zhang, Rongrong Ji, and Yue Gao. 2019. Hypergraph neural networks. In *Proceedings of the AAAI Conf. on Artificial Intelligence*, Vol. 33. 3558–3565.

[10] Xinyu Fu, Jiani Zhang, Ziqiao Meng, and Irwin King. 2020. Magnn: Metapath aggregated graph neural network for heterogeneous graph embedding. In *Proceedings of the Web Conference 2020*. 2331–2341.

[11] Yue Gao, Yifan Feng, Shuyi Ji, and Rongrong Ji. 2022. HGNN$^+$: General Hypergraph Neural Networks. *IEEE Trans. on Pattern Analysis and Machine Intelligence* (2022).

[12] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth Intl. Conf. on Artificial Intelligence and Statistics*. JMLR Workshop and Conference Proceedings, 249–256.

[13] Palash Goyal, Sujit Rokka Chhetri, and Arquimedes Canedo. 2020. dyngraph2vec: Capturing network dynamics using dynamic graph representation learning. *Knowledge-Based Systems* 187 (2020), 104816.

[14] Palash Goyal, Nitin Kamra, Xinran He, and Yan Liu. 2018. Dyngem: Deep embedding method for dynamic graphs. *arXiv preprint arXiv:1805.11273* (2018).

[15] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD Intl. Conf. on Knowledge Discovery and Data Mining*. 855–864.

[16] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems* 30 (2017).

[17] Malik Khizar Hayat, Shan Xue, Jia Wu, and Jian Yang. 2024. Heterogeneous Hypergraph Embedding for Node Classification in Dynamic Networks. *IEEE Trans. on Artificial Intelligence* (2024).

[18] Malik Khizar Hayat, Shan Xue, and Jian Yang. 2023. Self-supervised Heterogeneous Hypergraph Learning with Context-aware Pooling for Graph-level Classification. In *2023 IEEE Intl. Conf. on Data Mining (ICDM)*. IEEE, 140–149.

[19] Malik Khizar Hayat, Shan Xue, and Jian Yang. 2024. Self-supervised multi-hop heterogeneous hypergraph embedding with informative pooling for graph-level classification. *Knowledge and Information Systems* (2024), 1–24.

[20] Chenghan Huang, Lili Wang, Xinyuan Cao, Weicheng Ma, and Soroush Vosoughi. 2022. Learning dynamic graph embeddings using random walk with temporal backtracking. In *NeurIPS 2022 Temporal Graph Learning Workshop*.

[21] Byung-Hoon Kim, Jong Chul Ye, and Jae-Jin Kim. 2021. Learning dynamic graph representation of brain connectome with spatio-temporal attention. *Advances in Neural Information Processing Systems* 34 (2021), 4314–4327.

[22] Thomas N Kipf and Max Welling. 2016. Variational graph auto-encoders. (2016).

[23] Junhyun Lee, Inyeop Lee, and Jaewoo Kang. 2019. Self-attention graph pooling. In *Intl. Conf. on machine learning*. pmlr, 3734–3743.

[24] Mengran Li, Yong Zhang, Wei Zhang, Yi Chu, Yongli Hu, and Baocai Yin. 2023. Self-supervised Nodes-Hyperedges Embedding for Heterogeneous Information Network Learning. *IEEE Trans. on Big Data* (2023).

[25] Xiao Li, Li Sun, Mengjie Ling, and Yan Peng. 2023. A survey of GNN-based recommendation in social networks. *Neurocomputing* 549 (2023), 126–441.

[26] Zhijun Liu, Chao Huang, Yanwei Yu, and Junyu Dong. 2021. Motif-preserving dynamic attributed network embedding. In *Proceedings of the Web Conf. 2021*. 1629–1638.

[27] Zexi Liu, Bohan Tang, Ziyuan Ye, Xiaowen Dong, Siheng Chen, and Yanfeng Wang. 2024. Hypergraph transformer for semi-supervised classification. In *ICASSP 2024-2024 IEEE Intl. Conf. on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 7515–7519.

[28] Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).

[29] Annamalai Narayanan, Mahinthan Chandramohan, Rajasekar Venkatesan, Lihui Chen, Yang Liu, and Shantanu Jaiswal. 2017. graph2vec: Learning distributed representations of graphs. *arXiv preprint arXiv:1707.05005* (2017).

[30] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao Schardl, and Charles Leiserson. 2020. Evolvegcn: Evolving graph convolutional networks for dynamic graphs. In *Proceedings of the AAAI Conf. on Artificial Intelligence*, Vol. 34. 5363–5370.

[31] Jiezhong Qiu, Qibin Chen, Yuxiao Dong, Jing Zhang, Hongxia Yang, Ming Ding, Kuansan Wang, and Jie Tang. 2020. Gcc: Graph contrastive coding for graph neural network pre-training. In *Proceedings of the 26th ACM SIGKDD Intl. Conf. on Knowledge Discovery & Data Mining*. 1150–1160.

[32] Shota Saito. 2022. Hypergraph modeling via spectral embedding connection: Hypergraph cut, weighted kernel k-means, and heat kernel. In *Proceedings of the AAAI Conf. on Artificial Intelligence*, Vol. 36. 8141–8149.

[33] Wei Shao, Yao Peng, Chen Zu, Mingliang Wang, Daoqiang Zhang, Alzheimer's Disease Neuroimaging Initiative, et al. 2020. Hypergraph based multi-task feature selection for multimodal classification of Alzheimer's disease. *Computerized Medical Imaging and Graphics* 80 (2020), 101663.

[34] Gen Shi, Yifan Zhu, Jian K Liu, and Xuesong Li. 2023. Hegcl: Advance self-supervised learning in heterogeneous graph-level representation. *IEEE Trans. on Neural Networks and Learning Systems* (2023).

[35] Qingyun Sun, Jianxin Li, Hao Peng, Jia Wu, Yuanxing Ning, Philip S Yu, and Lifang He. 2021. Sugar: Subgraph neural network with reinforcement pooling and self-supervised mutual information mechanism. In *Proceedings of the Web Conference 2021*. 2081–2091.

[36] Yunbo Tang, Dan Chen, Jia Wu, Weiping Tu, Jessica JM Monaghan, Paul Sowman, and David Mcalpine. 2023. Functional connectivity learning via Siamese-based SPD matrix representation of brain imaging data. *Neural Networks* 163 (2023), 272–285.

[37] Sheng Tian, Ruofan Wu, Leilei Shi, Liang Zhu, and Tao Xiong. 2021. Self-supervised representation learning on dynamic graphs. In *Proceedings of the 30th ACM Intl. Conf. on Information & Knowledge Management*. 1814–1823.

[38] Daixin Wang, Peng Cui, and Wenwu Zhu. 2016. Structural deep network embedding. In *Proceedings of the 22nd ACM SIGKDD Intl. Conf. on Knowledge Discovery and Data Mining*. 1225–1234.

[39] Xiao Wang, Nian Liu, Hui Han, and Chuan Shi. 2021. Self-supervised heterogeneous graph neural network with co-contrastive learning. In *Proceedings of the 27th ACM SIGKDD Intl. Conf. on Knowledge Discovery & data mining*. 1726–1736.

[40] Xuemei Wei, Yezheng Liu, Jianshan Sun, Yuanchun Jiang, Qifeng Tang, and Kun Yuan. 2023. Dual subgraph-based graph neural network for friendship prediction in location-based social networks. *ACM Trans. on Knowledge Discovery from Data* 17, 3 (2023), 1–28.

[41] Zhihao Wen and Yuan Fang. 2022. Trend: Temporal event and node dynamics for graph representation learning. In *Proceedings of the ACM Web Conf. 2022*. 1159–1169.

[42] Lianghao Xia, Chao Huang, and Chuxu Zhang. 2022. Self-supervised hypergraph transformer for recommender systems. In *Proceedings of the 28th ACM SIGKDD Conf. on Knowledge Discovery and Data Mining*. 2100–2109.

[43] Li Xiao, Junqi Wang, Peyman H Kassani, Yipu Zhang, Yuntong Bai, Julia M Stephen, Tony W Wilson, Vince D Calhoun, and Yu-Ping Wang. 2019. Multi-hypergraph learning-based brain functional connectivity analysis in fMRI data. *IEEE Trans. on Medical Imaging* 39, 5 (2019), 1746–1758.

[44] Dongkuan Xu, Wei Cheng, Dongsheng Luo, Haifeng Chen, and Xiang Zhang. 2021. Infogcl: Information-aware graph contrastive learning. *Advances in Neural Information Processing Systems* 34 (2021), 30414–30425.

[45] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph contrastive learning with augmentations. *Advances in Neural Information Processing Systems* 33 (2020), 5812–5823.

[46] Haozhen Zhang, Xueting Han, Xi Xiao, and Jing Bai. 2023. Time-aware Graph Structure Learning via Sequence Prediction on Temporal Graphs. In *Proceedings of the 32nd ACM Intl. Conf. on Info. and Knowledge Management*. 3288–3297.

[47] Siwei Zhang, Yun Xiong, Yao Zhang, Yiheng Sun, Xi Chen, Yizhu Jiao, and Yangyong Zhu. 2023. RDGSL: Dynamic Graph Representation Learning with Structure Learning. In *Proceedings of the 32nd ACM Intl. Conf. on Info. and Knowledge Management*. 3174–3183.

[48] Yongqi Zhang, Quanming Yao, Ling Yue, Xian Wu, Ziheng Zhang, Zhenxi Lin, and Yefeng Zheng. 2023. Emerging drug interaction prediction enabled by a flow-based graph neural network with biomedical network. *Nature Computational Science* 3, 12 (2023), 1023–1033.

[49] Lekui Zhou, Yang Yang, Xiang Ren, Fei Wu, and Yueting Zhuang. 2018. Dynamic network embedding by modeling triadic closure process. In *Proceedings of the AAAI Conf. on Artificial Intelligence*, Vol. 32. 571–578.